

# 포팅 매뉴얼

- 1. 프로젝트 개요
- 2. 개발 환경
  - 2.1 Frontend
  - 2.2 Backend
  - 2.3 Infra
  - 2.4 AI
  - 2.5 Embadded
  - 2.6 Database
  - 2.7 형상/이슈관리
- 3. 환경 변수 및 세부 설정
  - 3.1 Frontend
  - 3.2 Backend
  - 3.3 AI
- 4. 빌드 및 배포
  - 4.1 인프라 및 데이터베이스
  - 4.2 백엔드 서버
  - 4.3 AI 서버
  - 4.4 프론트엔드

## 1. 프로젝트 개요

- 프로젝트명: Orbit
- 프로젝트 한 줄 설명: 주행형 홈 AIoT 로봇 기반의 일과 기록, 요약 리포트 서비스
- 구조

```
/
├ BE/      # FastAPI 메인 서버
├ AI/      # AI 서버
├ EMB/     # HW 제어
├ FE/      # React Native App
└ 기타 설정 # docker-compose, infra 설정
```

## 2. 개발 환경

### 2.1 Frontend

- Node.js 24
- Expo SDK 54.x
- React / React Native 19.1.0 / 0.81.5
- TypeScript 5.9.x
- Zustand 5.0.x
- React Navigation 7.x (native, stack, bottom-tabs)
- react-native-sse 1.2.x
- expo-font 14.x
- react-hook-form / yup 7.x / 1.7.x

### 2.2 Backend

- Python 3.10
- FastAPI

## 2.3 Infra

- AWS EC2
- Nginx
- Jenkins
- Docker
- Docker Compose
- Mosquitto MQTT

## 2.4 AI

- Python 3.10
- FastAPI
- Torch 2.8.0 (+cu128)
- google-gemini(Gemini 2.5 flash-lite)

## 2.5 Embadded

- Python 3.10
- C++
- Linux
- NVIDIA Jetson Orin Nano
- RPi5
- STM32F401RE

## 2.6 Database

- PostgreSQL 16.11
- AWS S3

## 2.7 형상/이슈관리

- Gitlab
- Jira

# 3. 환경 변수 및 세부 설정

## 3.1 Frontend

```
EXPO_PUBLIC_API_BASE_URL=
```

## 3.2 Backend

```
# AI SERVER
AI_BASE_URL=
AI_TIMEOUT_SEC=
```

```
# DB
POSTGRES_DB=
POSTGRES_USER=
POSTGRES_PASSWORD=
DATABASE_URL=postgresql+asyncpg://...
TZ=Asia/Seoul

# MQTT
MQTT_BROKER=
MQTT_PORT=
MQTT_USERNAME=
MQTT_PASSWORD=

# JWT
JWT_SECRET_KEY=
JWT_ALGORITHM=
ACCESS_TOKEN_EXPIRE_MINUTES=
REFRESH_TOKEN_EXPIRE_DAYS=

# AWS S3
S3_BUCKET=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_REGION=
```

### 3.3 AI

```
GMS_API_KEY=

device: cuda
cuda_visible_devices: "0"

llm_params:
  llm_model_name: "gemini-2.5-flash-lite"
  base_url: "https://generativelanguage.googleapis.com"
```

## 4. 빌드 및 배포

- 인프라/DB(기반) → 백엔드(코어) → AI(기능) → 프론트엔드(클라이언트) 순으로 진행

### 4.1 인프라 및 데이터베이스

- 백엔드 구동 전, 데이터베이스와 메시지 브로커가 선행되어야 합니다.
- **실행 (Docker Compose)**
  - 프로젝트 루트에서 실행.

```
# DB, MQTT, Nginx 컨테이너 구동
docker compose up -d postgres mosquitto npm
```

- **DB 초기화 확인**
  - `BE/database/init.sql` 파일이 마운트되어 컨테이너 구동 시 `orbit` 스키마 및 `ENUM` 타입 자동 생성됩니다.

- 확인: `docker compose logs postgres`

## 4.2 백엔드 서버

- 사전 설정
  - `BE/.env` 파일 존재 여부 및 `JWT_SECRET_KEY` 설정 확인.
  - `S3` 및 `AI_BASE_URL` 환경 변수 확인.
- 빌드 및 실행

```
docker compose up -d --build be-api
```

## 4.3 AI 서버

- 환경 설정
  - `AI/server/.env` : `GMS_API_KEY` (Gemini API) 설정 필수.
  - `AI/server/config.yaml` : `device: cuda` (GPU 사용 시) 설정 확인.
- 의존성 설치 및 실행

```
cd AI/server

# 가상환경 생성 및 활성화
python -m venv .venv
source .venv/bin/activate # Windows: .\.venv\Scripts\activate

# 라이브러리 설치
pip install -r requirements.txt

# 서버 실행 (포트 충돌 방지 주의)
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

## 4.4 프론트엔드

- 환경 변수 주입
  - `FE/.env` 파일 내 `EXP0_PUBLIC_API_BASE_URL` 에 백엔드 주소 기입.
- 의존성 설치

```
cd FE
npm install
```

- 로컬 실행

```
npx expo start --web
```